Lab 4 –Pipes

Problem 1:

Write a C program that forks a child and uses a pipe for child and parent communication

1. The child should read a string from the user and store it in a local array of characters
2. The child should then send the string to the parent using the pipe and display the string and the number of bytes sent
3. The parent process should read the string and display the string and the number of bytes received.
4. The parent should then count the number of characters received and send the number back to the child which will compare it with the length of the message transmitted and hence, confirm that the message was well received

Sample Output:

```
Enter a string:
Hello
Child wrote Hello
Child wrote 5 bytes
Parent read Hello
Parent read 5 bytes
Parent length received=5
Child: Length received from parent=5
Child:  Length transmitted=5
Message successfully received!!
```

Problem 2:

Write a C program that forks a child and uses pipe as mean of IPC to implement the following command: “ pwd | wc – c”

- The child should execute the “pwd” command and send the output to the parent
- The parent should execute the “wc -c” command and returns the result
- Use dup2() for redirecting the standard input and output

Problem 3:

Write a C program that forks a child and uses a pipe for child and parent communication

- The child should:
  1. Read characters from the parents one by one and count them
  2. Return the number of characters read
- The parent should:
  1. Send characters one by one to the child
  2. Return the number of characters read. The characters are taken from the command line argument (should handle multiple arguments)
  3. Wait for the child to terminate and read the number of characters read (returned by the child process).
  4. Print the number of characters read by the child

Sample input/output:

```
$ ./test Hello
Child Read = 5
Child count received 5 chars
```